

查表法

在編解碼系統中，運算效能偏低的處理器架構可以利用查表法去節省不少的運算時間。以下為一種 8-bit 編碼表，該表總共有 256 個編號(0 至 255)，每個編號都會對應 1 個 8-bit 正整數；例如編號 4 對應的數值為 196。假設你有 m 筆 8-bit 正整數輸入資料欲進行編碼，試撰寫一程式能根據以下演算法，去計算最後的 checksum。

- (1) checksum 的初始值為 0。
- (2) checksum 跟 8-bit 正整數輸入值進行 xor 運算得出 B。
- (3) 根據以下編碼表，找尋編號 B 對應的 8-bit 數值，並取代之前的 checksum，以進行下次的 xor 運算。
- (4) 重複步驟(2)和(3)，直到輸入最後一筆資料，並利用編碼表找出最後要取代 checksum 的數值。
- (5) 最後 checksum 需要進行反轉(亦即 bit 0 = bit 7, bit 1 = bit 6, bit 2 = bit 5, bit 3 = bit 4)，反轉後的數字為最終答案。

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	49	98	83	196	245	166	151	185	136	219	234	125	76	31	46
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
67	114	33	16	135	182	229	212	250	203	152	169	62	15	92	109
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
134	183	228	213	66	115	32	17	63	14	93	108	251	202	153	168
48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
197	244	167	150	1	48	99	82	124	77	30	47	184	137	218	235
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
61	12	95	110	249	200	155	170	132	181	230	215	64	113	34	19
80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
126	79	28	45	186	139	216	233	199	246	165	148	3	50	97	80
96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
187	138	217	232	127	78	29	44	2	51	96	81	198	247	164	149
112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
248	201	154	171	60	13	94	111	65	112	35	18	133	180	231	214
128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
122	75	24	41	190	143	220	237	195	242	161	144	7	54	101	84
144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
57	8	91	106	253	204	159	174	128	177	226	211	68	117	38	23
160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
252	205	158	175	56	9	90	107	69	116	39	22	129	176	227	210
176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
191	142	221	236	123	74	25	40	6	55	100	85	194	243	160	145
192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
71	118	37	20	131	178	225	208	254	207	156	173	58	11	88	105
208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
4	53	102	87	192	241	162	147	189	140	223	238	121	72	27	42
224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239
193	240	163	146	5	52	103	86	120	73	26	43	188	141	222	239
240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255
130	179	224	209	70	119	36	21	59	10	89	104	255	206	157	172

輸入說明：

第 1 行輸入 m 筆 8 位元資料，其範圍為 $1 \leq m \leq 100$ 。

第 2 行輸入 m 筆 8 位元長度的整數 n ，其中 $0 \leq n \leq 255$ 。

輸出說明：

輸出反轉後的 checksum。

範例說明：

範例 1 輸入	範例 1 輸出
1 206	26
範例 2 輸入	範例 2 輸出
3 152,7,9	160

Lookup Table Method (LUT)

In encoding and decoding systems, processor architectures with lower computing performance can utilize the lookup table method to save a significant amount of computation time. The following is an 8-bit encoding table containing a total of 256 index numbers (0 to 255), where each index number corresponds to an 8-bit positive integer; for example, index 4 corresponds to the value 196. Assuming you have m pieces of 8-bit positive integer input data to be encoded, write a program to calculate the final checksum based on the following algorithm.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	49	98	83	196	245	166	151	185	136	219	234	125	76	31	46
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
67	114	33	16	135	182	229	212	250	203	152	169	62	15	92	109
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
134	183	228	213	66	115	32	17	63	14	93	108	251	202	153	168
48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
197	244	167	150	1	48	99	82	124	77	30	47	184	137	218	235
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
61	12	95	110	249	200	155	170	132	181	230	215	64	113	34	19
80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
126	79	28	45	186	139	216	233	199	246	165	148	3	50	97	80
96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
187	138	217	232	127	78	29	44	2	51	96	81	198	247	164	149
112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
248	201	154	171	60	13	94	111	65	112	35	18	133	180	231	214
128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
122	75	24	41	190	143	220	237	195	242	161	144	7	54	101	84
144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
57	8	91	106	253	204	159	174	128	177	226	211	68	117	38	23
160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
252	205	158	175	56	9	90	107	69	116	39	22	129	176	227	210
176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
191	142	221	236	123	74	25	40	6	55	100	85	194	243	160	145
192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
71	118	37	20	131	178	225	208	254	207	156	173	58	11	88	105
208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
4	53	102	87	192	241	162	147	189	140	223	238	121	72	27	42
224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239
193	240	163	146	5	52	103	86	120	73	26	43	188	141	222	239
240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255
130	179	224	209	70	119	36	21	59	10	89	104	255	206	157	172

- (1) The initial value of the checksum is 0.
- (2) Perform an XOR operation between the checksum and the 8-bit positive integer input value to obtain B.
- (3) Based on the encoding table below, look up the 8-bit value corresponding to index B and use it to replace the previous checksum for the next XOR operation.
- (4) Repeat steps (2) and (3) until the last piece of data is inputted, and use the encoding table to find the final value to replace the checksum.
- (5) Finally, the checksum needs to be reversed (i.e., bit 0 = bit 7, bit 1 = bit 6, bit 2 = bit 5, bit 3 = bit 4). The reversed number is the final answer.

Input Description:

The 1st line inputs m pieces of 8-bit data, where the range is $1 \leq m \leq 100$. The 2nd line inputs m 8-bit long integers n , where $0 \leq n \leq 255$.

Output Description:

Output the reversed checksum.

Example Description:**Example 1**

Input

1

206

Output 26

Example 2

Input

3

152,7,9

Output

160